```
numberSold = 0

calculateNumberSold = (salePrice) ->
  numberSold = 50 + 20/10 * (200 - salePrice)

calculateRevenue = (salePrice, callback) ->
  callback numberSold * salePrice
revenueBetween = (start, finish) ->
  totals = []
  for price in [start..finish]
    calculateNumberSold price
    addToTotals = (result) ->
      totals.push result
    calculateRevenue price, addToTotals
  totals

revenueBetween 140, 145
# [ 23800, 23688, 23572, 23452, 23328, 23200 ]
```

numberSold is shared state.

totals is local (to the function) state.

In listing 6.3 the `totals` variable is local to the `revenueBetween` function; other parts of the program can't assign a value to it. On the other hand, `numberSold` is shared by all of the functions in the program because they're all part of the same scope. This can have disastrous consequences.

Now imagine that `calculateRevenue` from listing 6.3 takes some time before it invokes the callback because it has to wait for a database. Approximate this with a `setTimeout` call:

```
oneSecond = 1000
calculateRevenue = (callback) ->
  setTimeout ->
    callback numberSold * salePrice
  , oneSecond
```

You're surprised when you find out what `revenueBetween` returns:

```
revenueBetween 140, 145
# []
```

You kept the totals in an array, assigned values to them, and returned the result. This imperative solution needed things done in a particular order but didn't enforce that order—a move to asynchronous and the solution broke.